Part 1: Design Pattern Identification

For each scenario below, identify the most appropriate design pattern and explain why in 2-3 sentences.

Question 1

Scenario:
You are developing an image processing application that supports different filters (e.g., grayscale, sepia, blur, etc.). Each filter is applied to the image, but the number of filters will grow in the future. You need a way to apply filters dynamically without changing the existing code.
Which design pattern would you use?

Question 2

Scenario:
You are designing a web-based multiplayer game. Every player should have access to the same game state across multiple sessions, but only one instance of the game state should be allowed. You need to ensure that all players are connected to the same server instance to prevent data inconsistency.
Which design pattern would you use?

Part 2: Apply Patterns

Scenario 1 – Apply Factory Method

Problem:
You are building an app to handle customer service in a retail store. Initially, the system only supports handling in-store queries, but now you need to add support for online and phone queries as well. The system should allow these types of queries to be handled without changing existing code.
Task:
Write Java code implementing the Factory Method pattern.

Scenario 2 – Apply Abstract Factory

Problem:
You are building an online store that sells electronics. The store offers two main product families:

- Mobile Devices (Smartphones, Tablets, Smartwatches)
- Laptops (Gaming, Business, and Ultrabooks)

Each product category has different methods for "checkBattery()" (for Mobile Devices) and "runDiagnostics()" (for Laptops). The client should be able to choose which product family to work with at runtime.
Task:
Implement the Abstract Factory pattern for this scenario.

Scenario 3 – Identify & Apply

Problem:
You are building a weather tracking system that notifies users whenever there is a change in weather conditions. Various clients (e.g., mobile app, website, email) need to receive these updates in real-time. The weather system should notify all clients whenever the weather data changes, without knowing how each client will use the data.
Question:
Which design pattern would you use to solve this problem?
Question: Write Code in Java.

Scenario 4 – Identify & Apply

Problem:
You are designing a customizable meal plan app. The basic meal includes rice, vegetables, and a protein source. Users can customize their meal by adding extra ingredients such as nuts, fruits, or sauces. Each addition alters the price of the meal and updates the meal list. The system should handle thousands of custom meal combinations.
Question:
Which design pattern would you use to solve this problem?
Question: Also identify which Design principle will be used.
Question: Write Code in Java.

Scenario 5(A) – Fix the Code By Applying Singleton Pattern

Problem:
A ride-sharing app has a RideManager class that assigns drivers to users. Multiple users are requesting rides simultaneously. However, multiple instances of RideManager are being created, causing errors in the ride assignment logic. Only one instance of RideManager should exist.
Buggy Code (No Singleton Applied):

```
class RideManager {
    private Set<Integer> assignedRides = new HashSet<>();

    public RideManager() {
        System.out.println("New RideManager created!");
    }

    public void assignRide(String userName) {
        int rideNumber = assignedRides.size() + 1;
        assignedRides.add(rideNumber);
        System.out.println(userName + " assigned ride #" + rideNumber);
    }

    public void showAssignedRides() {
        System.out.println("Assigned rides: " + assignedRides);
```

```
    }
}

public class BuggyRideSharingApp {
    public static void main(String[] args) {
        RideManager user1 = new RideManager();
        user1.assignRide("User1");
        user1.assignRide("User2");

        RideManager user2 = new RideManager();
        user2.assignRide("User3");

        System.out.println("\n--- PROBLEM ---");
        user1.showAssignedRides();
        user2.showAssignedRides();
    }
}
```

Expected Output:
Only one instance of `RideManager` should assign rides, ensuring no duplication across users.

Buggy Code Example 5 (B)– Not Using Observer Pattern for Notifications

Scenario:
An online auction platform notifies users about the status of their bids, but the notifications are directly tied to the auction logic, making it difficult to add new ways to notify users (e.g., email, SMS, push notifications).

Buggy Code (No Observer Pattern Applied):

```
class Auction {
    private String currentBidder;

    public void placeBid(String user) {
        currentBidder = user;
        System.out.println("Bid placed by: " + user);
        // Notify user
        sendBidNotification(user);
    }

    private void sendBidNotification(String user) {
        System.out.println("Notifying user: " + user);
    }
}

public class BuggyAuctionSystem {
    public static void main(String[] args) {
        Auction auction = new Auction();
        auction.placeBid("John");
    }
}
```

Problem:

- The `Auction` class directly handles notifications, which makes it hard to change the notification method or add new notification channels.